Database Systems Lab — Week 02 Manual

SQL Fundamentals with Real Datasets

Course: Database Systems Lab (Data Science Program)

Duration: 3 hours (2 lab sessions)

Prerequisites: Lab 01 complete — PostgreSQL running, DBeaver connected, GitHub repo initialized

---

Learning Objectives

By the end of this lab you will be able to:

- Set up a multi-table database from a SQL script using both psql and DBeaver
- Write SELECT statements to retrieve and compute columns
- Filter rows with WHERE using comparisons, ranges, patterns, and NULL checks
- Sort and limit results with ORDER BY and LIMIT
- Understand common PostgreSQL data types and the errors they cause when misused
- Read a basic EXPLAIN ANALYZE output
- Commit your SQL work to GitHub using the workflow from Lab 01

Why SQL? The Data Science Connection

Before you train a model, build a dashboard, or run any analysis, you need data — correctly filtered, shaped, and understood. SQL is the universal language for that job. In most data science roles, analysts write more SQL per day than they write Python.

Here is where today's work fits into a real pipeline:

Stage	What you need SQL for	Clauses today
Exploration	Understand what a table contains	SELECT, LIMIT
Filtering	Isolate relevant records	WHERE
Feature extraction	Pull and compute columns	SELECT with expressions
Reporting	Ranked or top-N output	ORDER BY, LIMIT

The dataset you will build today is a simplified version of what you would find at a real e-commerce company. Every query you write this week is the kind of thing a data analyst or data scientist writes on the first day of a new project.

Part 1 — Dataset Setup

Before writing any queries you need data to query. You will create a small but realistic e-commerce database: four related tables, real-looking records, and enough variety to make every exercise meaningful.

What the schema looks like:

- **customers** — who buys from the store
- **products** — what the store sells
- **orders** — each purchase transaction
- **order_items** — the individual products inside each order

This is a simplified version of the kind of schema you will design yourself in Week 4.

Step 1.1 — Open Your Database

Open Ubuntu terminal and connect to PostgreSQL. If you get a "connection refused" error, start the service first.

```
# Start PostgreSQL if it is not running (WSL/Linux users)
sudo service postgresql start

# Connect to your lab database
psql -U your_username -d lab1_db
```

macOS users:

```
brew services start postgresql@15
psql lab1_db
```

You should see your database prompt: `lab1_db=#`

Step 1.2 — Create the Dataset File

Open a second terminal tab (keep psql open in the first). Navigate to your course folder and create a new file for this week.

```
cd ~/database-labs
mkdir lab2
cd lab2
nano ecommerce_setup.sql
```

Paste the entire script below into the nano editor, then save with Ctrl+O, Enter, Ctrl+X.

```
-- ============================================================
-- Lab 02: E-Commerce Dataset
-- Author: [Your Name]
-- Date:   [Today's Date]
-- ============================================================

-- Drop tables if they exist (clean slate)
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS customers;

-- ── CUSTOMERS ──────────────────────────────────────────────
CREATE TABLE customers (
    customer_id  SERIAL PRIMARY KEY,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(150) UNIQUE NOT NULL,
    city         VARCHAR(80),
    signup_date  DATE NOT NULL
);

-- ── PRODUCTS ───────────────────────────────────────────────
CREATE TABLE products (
    product_id   SERIAL PRIMARY KEY,
    product_name VARCHAR(150) NOT NULL,
    category     VARCHAR(60)  NOT NULL,
    price        DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_qty    INTEGER NOT NULL DEFAULT 0
);

-- ── ORDERS ─────────────────────────────────────────────────
CREATE TABLE orders (
    order_id      SERIAL PRIMARY KEY,
    customer_id   INTEGER NOT NULL REFERENCES customers(customer_id),
    order_date    DATE NOT NULL,
    status        VARCHAR(30) NOT NULL DEFAULT 'pending',
    total_amount  DECIMAL(10,2) NOT NULL,
    shipped_date  DATE
);

-- ── ORDER ITEMS ────────────────────────────────────────────
CREATE TABLE order_items (
    item_id     SERIAL PRIMARY KEY,
    order_id    INTEGER NOT NULL REFERENCES orders(order_id),
    product_id  INTEGER NOT NULL REFERENCES products(product_id),
    quantity    INTEGER NOT NULL CHECK (quantity > 0),
    unit_price  DECIMAL(10,2) NOT NULL
);
```

```
-- ═══════════════════════════════════════════════════════════
-- INSERT DATA
-- ═══════════════════════════════════════════════════════════

INSERT INTO customers (name, email, city, signup_date) VALUES
('Ali Hassan',        'ali.hassan@gmail.com',       'Peshawar',   '2023-03-12'),
('Sara Khan',         'sara.khan@yahoo.com',         'Lahore',     '2023-05-
20'),
('Usman Tariq',       'usman.tariq@gmail.com',       'Karachi',    '2023-07-
08'),
('Fatima Malik',      'fatima.malik@hotmail.com',    'Islamabad',  '2023-08-
15'),
('Bilal Ahmed',       'bilal.ahmed@gmail.com',       'Peshawar',   '2023-09-
01'),
('Ayesha Noor',       'ayesha.noor@gmail.com',       'Lahore',     '2023-10-
10'),
('Hamza Sheikh',      'hamza.sheikh@outlook.com',    'Karachi',    '2023-11-
22'),
('Zainab Iqbal',      'zainab.iqbal@gmail.com',      'Quetta',     '2024-01-
05'),
('Farhan Butt',       'farhan.butt@yahoo.com',       'Lahore',     '2024-02-
14'),
('Sana Rashid',       'sana.rashid@gmail.com',       'Islamabad',  '2024-03-
20'),
('Omar Hussain',      'omar.hussain@gmail.com',      'Peshawar',   '2024-04-
11'),
('Nadia Aziz',        'nadia.aziz@hotmail.com',      'Karachi',    '2024-05-
30'),
('Tariq Mehmood',     'tariq.mehmood@gmail.com',     'Lahore',     '2024-06-
18'),
('Hina Qureshi',      'hina.qureshi@yahoo.com',      'Islamabad',  '2024-07-
25'),
('Kamran Javed',      'kamran.javed@gmail.com',      'Faisalabad', '2024-08-
09'),
('Rabia Siddiqui',    'rabia.siddiqui@gmail.com',    'Karachi',    '2024-09-
14'),
('Imran Chaudhry',    'imran.chaudhry@outlook.com',  'Lahore',     '2024-10-
03'),
('Maira Aslam',       'maira.aslam@gmail.com',       'Peshawar',   '2024-11-
19'),
('Asad Nawaz',        'asad.nawaz@gmail.com',        'Multan',     '2024-12-
01'),
('Kiran Baig',        'kiran.baig@yahoo.com',        'Karachi',    '2025-01-
07');

INSERT INTO products (product_name, category, price, stock_qty) VALUES
('Wireless Mouse',           'Electronics',   1850.00,  45),
('USB-C Hub 7-in-1',         'Electronics',   3200.00,  30),
('Mechanical Keyboard',      'Electronics',   7500.00,  20),
('Laptop Stand Aluminium',   'Electronics',   4200.00,  35),
('Noise Cancelling Headset', 'Electronics',   12500.00, 15),
('Python Programming Book',  'Books',         1200.00,  60),
('Data Science Handbook',    'Books',         1800.00,  40),
('Machine Learning Guide',   'Books',         2200.00,  25),
('Database Design Patterns', 'Books',         1600.00,  30),
('Clean Code',               'Books',         1400.00,  50),
('Office Chair Ergonomic',   'Furniture',     28000.00,  8),
('Standing Desk Converter',  'Furniture',     15000.00, 12),
('Monitor Arm Dual',         'Furniture',     8500.00,  18),
('Webcam 1080p',             'Electronics',   5500.00,  22),
('LED Desk Lamp',            'Electronics',   2800.00,  40);
```

```
INSERT INTO orders (customer_id, order_date, status, total_amount, shipped_date)
VALUES
(1,  '2024-09-05',  'delivered',   3050.00,  '2024-09-07'),
(2,  '2024-09-12',  'delivered',   8700.00,  '2024-09-14'),
(3,  '2024-10-01',  'delivered',  12500.00,  '2024-10-03'),
(4,  '2024-10-15',  'delivered',   4200.00,  '2024-10-17'),
(5,  '2024-11-02',  'delivered',   2400.00,  '2024-11-04'),
(6,  '2024-11-18',  'delivered',  36500.00,  '2024-11-21'),
(7,  '2024-12-05',  'delivered',   7500.00,  '2024-12-07'),
(8,  '2024-12-20',  'delivered',   3400.00,  '2024-12-22'),
(9,  '2025-01-08',  'delivered',   9700.00,  '2025-01-10'),
(10, '2025-01-22',  'delivered',   5500.00,  '2025-01-24'),
(11, '2025-02-03',  'delivered',  15000.00,  '2025-02-05'),
(12, '2025-02-14',  'cancelled',   1200.00,   NULL),
(13, '2025-03-01',  'delivered',  21700.00,  '2025-03-03'),
(14, '2025-03-10',  'shipped',     6400.00,  '2025-03-11'),
(15, '2025-03-22',  'processing',  2800.00,   NULL),
(1,  '2025-04-02',  'delivered',  13700.00,  '2025-04-04'),
(3,  '2025-04-15',  'delivered',   4400.00,  '2025-04-17'),
(5,  '2025-05-01',  'processing',  7500.00,   NULL),
(16, '2025-05-08',  'pending',     1800.00,   NULL),
(17, '2025-05-12',  'delivered',  28000.00,  '2025-05-14'),
(18, '2025-05-20',  'shipped',     5500.00,  '2025-05-21'),
(19, '2025-06-01',  'delivered',   3600.00,  '2025-06-03'),
(20, '2025-06-10',  'pending',    12500.00,   NULL),
(2,  '2025-06-18',  'delivered',   2200.00,  '2025-06-20'),
(8,  '2025-07-05',  'processing',  8500.00,   NULL),
(10, '2025-07-12',  'delivered',  15000.00,  '2025-07-14'),
(12, '2025-07-20',  'delivered',   4600.00,  '2025-07-22'),
(14, '2025-08-02',  'pending',     1600.00,   NULL),
(6,  '2025-08-15',  'delivered',  43000.00,  '2025-08-17'),
(9,  '2025-08-28',  'shipped',     9700.00,  '2025-08-29');

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1,  1,  1, 1850.00), (1,  6,  1, 1200.00),
(2,  3,  1, 7500.00), (2,  7,  1, 1200.00),
(3,  5,  1,12500.00),
(4,  4,  1, 4200.00),
(5,  1,  1, 1850.00), (5,  10, 1, 1400.00),
(6,  11, 1,28000.00), (6,  4,  1, 4200.00), (6,  2,  1, 3200.00), (6, 13, 1,
1100.00),
(7,  3,  1, 7500.00),
(8,  8,  1, 2200.00), (8,  10, 1, 1200.00),
(9,  5,  1,12500.00), (9,  2,  1, 3200.00), (9,  6,  2, 2000.00),  -- note:
rounding on total
(10, 14, 1, 5500.00),
(11, 12, 1,15000.00),
(12, 6,  1, 1200.00),
(13, 11, 1,28000.00), (13, 2,  1, 3200.00),
(14, 4,  1, 4200.00), (14, 15, 1, 2200.00),
(15, 15, 1, 2800.00),
(16, 5,  1,12500.00), (16, 7,  1, 1200.00),
(17, 4,  1, 4200.00), (17, 9,  1,  200.00),
(18, 3,  1, 7500.00),
(19, 16, 1, 1800.00), (19, 19, 1, 1800.00),
(20, 23, 1,12500.00),
(21, 14, 1, 5500.00),
(22, 22, 1, 3600.00),
(23, 8,  1, 2200.00),
(24, 25, 1, 8500.00),
```

```
(25, 12, 1,15000.00),
(26, 1,  1, 1850.00), (26, 15, 1, 2800.00),
(27, 9,  1, 1600.00),
(28, 11, 1,28000.00), (28, 12, 1,15000.00),
(29, 5,  1, 9700.00);
```

Step 1.3 — Load the Script

Go back to your psql terminal and run the file.

```
\i /home/your_username/database-labs/lab2/ecommerce_setup.sql
```

You should see a series of CREATE TABLE and INSERT messages. If you see any errors, check that you saved the file correctly. Then verify the tables were created:

```
-- List all tables
\dt

-- Quick check — how many rows in each table?
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM order_items;
```

You should get 20, 15, 30, and a number of order_items rows respectively.

Step 1.4 — Explore the Tables in DBeaver

Open DBeaver. You should already have your lab1_db connection from Week 1. If you lost it or DBeaver cannot connect, fix the connection first using the troubleshooting steps from Lab 01.

In the left sidebar: expand your connection → Databases → lab1_db → Schemas → public → Tables.

You should now see four tables: customers, orders, order_items, products.

Right-click on **customers** → View Data. Spend two minutes looking at the data. Notice the column names, what kind of values are in each column, and whether any columns have nulls.

Do the same for the other three tables. You will write queries against this data for the rest of the lab, so understanding what it contains before you query it is not optional — it is the first thing any analyst does.

Step 1.5 — Commit the Setup File

Before touching any queries, commit your work to GitHub. This is the habit you are building: commit often, with meaningful messages.

```
# In your Ubuntu terminal (not psql)
cd ~/database-labs
git add lab2/ecommerce_setup.sql
git commit -m "Lab 2: Created e-commerce schema with seed data"
git push origin main
```

Part 2 — Core Concepts

Read each section, then try the examples in your psql terminal or DBeaver SQL editor before moving to the exercises.

2.1 The SELECT Statement

SELECT tells the database which columns to return. The FROM clause tells it which table.

```
-- Explore every column (fine for quick inspection, not for production)
SELECT * FROM customers;

-- Return only the columns you actually need
SELECT customer_id, name, city FROM customers;

-- Rename columns with AS for cleaner output
SELECT customer_id    AS "ID",
       name           AS "Customer Name",
       city           AS "City"
FROM customers;

-- Compute values directly in SELECT
SELECT product_name,
       price,
       price * 0.85          AS discounted_price,
       price * 0.15          AS discount_amount
FROM products;
```

Important habit: Avoid SELECT * on large tables and in any code you plan to keep. Listing columns explicitly makes your intent clear and avoids pulling data you do not need.

2.2 WHERE — Filtering Rows

WHERE filters which rows come back. The database evaluates the condition for every row and only returns rows where it is true.

```
-- Exact match
SELECT * FROM orders WHERE status = 'delivered';

-- Numeric comparison
SELECT * FROM products WHERE price > 5000;

-- Multiple conditions — ALL must be true
SELECT * FROM orders
WHERE status = 'delivered'
  AND total_amount > 10000;

-- Multiple conditions — ANY can be true
SELECT * FROM orders
WHERE status = 'pending'
   OR status = 'processing';

-- Cleaner way to do multiple OR on the same column
SELECT * FROM orders
WHERE status IN ('pending', 'processing', 'shipped');

-- Range (BETWEEN is inclusive on both ends)
SELECT * FROM products WHERE price BETWEEN 1000 AND 5000;

-- Pattern matching — % means "any characters"
SELECT * FROM customers WHERE email LIKE '%@gmail.com';

-- NULL checks — never use = NULL, always IS NULL
SELECT * FROM orders WHERE shipped_date IS NULL;
```

WHERE operator reference:

Operator	Meaning	Example
=	Exact match	WHERE status = 'delivered'
!= or <>	Not equal	WHERE status != 'cancelled'
> / >=	Greater than / or equal	WHERE price >= 5000
< / <=	Less than / or equal	WHERE stock_qty < 10
BETWEEN	Inclusive range	WHERE price BETWEEN 1000 AND 5000
IN (...)	Match any in list	WHERE city IN ('Lahore', 'Karachi')
LIKE	Pattern with % wildcard	WHERE email LIKE '%@gmail.com'
ILIKE	Case-insensitive LIKE	WHERE name ILIKE '%ali%'
IS NULL	No value present	WHERE shipped_date IS NULL
IS NOT NULL	Value is present	WHERE shipped_date IS NOT NULL

2.3 ORDER BY and LIMIT

Databases return rows in no guaranteed order unless you ask. ORDER BY sorts results. LIMIT cuts the output to a fixed number of rows.

```
-- Sort ascending (default — lowest to highest)
SELECT product_name, price FROM products ORDER BY price;

-- Sort descending (highest to lowest)
SELECT product_name, price FROM products ORDER BY price DESC;

-- Sort by multiple columns
-- "Newest orders first; when dates are equal, highest amount first"
SELECT order_id, customer_id, order_date, total_amount
FROM orders
ORDER BY order_date DESC, total_amount DESC;

-- Top 5 most expensive products
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;

-- Preview any new table — always do this first
SELECT * FROM orders LIMIT 10;
```

Critical habit: Any time you encounter a table you have not seen before, your first query should always be `SELECT * FROM table_name LIMIT 10`. Running a bare SELECT * on an unknown table without LIMIT is asking for trouble as datasets grow.

2.4 Data Types and the Errors They Cause

Every column has a type. Mixing it up in a WHERE clause causes errors or — worse — returns wrong results silently.

PostgreSQL type	Stores	Example value	Filter syntax
INTEGER / BIGINT	Whole numbers	42	WHERE stock_qty > 10
DECIMAL(10,2)	Exact decimals	19.99	WHERE price < 5000.00
VARCHAR / TEXT	Text	'Lahore'	WHERE city = 'Lahore'
DATE	Calendar date	'2025-01-15'	WHERE order_date > '2025-01-01'
BOOLEAN	True or false	TRUE	WHERE is_active = TRUE
SERIAL	Auto-increment ID	1, 2, 3	WHERE order_id = 7

The most common beginner error with data types:

```
-- WRONG — PostgreSQL reads this as arithmetic: 2025 minus 1 minus 15 = 2009
SELECT * FROM orders WHERE order_date > 2025-01-15;

-- CORRECT — wrap date values in single quotes
SELECT * FROM orders WHERE order_date > '2025-01-15';

-- WRONG — double quotes are for column/table names, not string values
SELECT * FROM customers WHERE city = "Lahore";

-- CORRECT
SELECT * FROM customers WHERE city = 'Lahore';
```

2.5 Clause Execution Order

SQL clauses do not run in the order you write them. PostgreSQL executes them in this sequence:

Step	Clause	What it does
1	FROM	Identify the table
2	WHERE	Filter rows
3	SELECT	Choose and compute columns
4	ORDER BY	Sort the result
5	LIMIT	Cut to final row count

Why does this matter? Because WHERE runs before SELECT, you cannot use an alias you defined in SELECT inside a WHERE clause. This is a common source of confusion:

```
-- WRONG — discounted_price does not exist yet when WHERE runs
SELECT product_name, price * 0.85 AS discounted_price
FROM products
WHERE discounted_price < 3000;

-- CORRECT — repeat the expression in WHERE
SELECT product_name, price * 0.85 AS discounted_price
FROM products
WHERE price * 0.85 < 3000;
```

Part 3 — Lab Exercises

Work through these in order. Each query builds on what came before. Use psql for typing queries and DBeaver to visualise results — practice both tools, because professional environments use both. Save every query you write to a file called `queries.sql` in your lab2 folder.

Query 1 — Explore the data

Before querying anything, understand what you have. Run each of these and spend 30 seconds looking at the results.

```
SELECT * FROM customers LIMIT 5;
SELECT * FROM products LIMIT 5;
SELECT * FROM orders LIMIT 5;
SELECT * FROM order_items LIMIT 5;
```

Write down in your lab notes: What does each table contain? What seems to be the relationship between orders and customers? What is the shipped_date column in orders — why are some values NULL?

Query 2 — Select specific columns

Return each customer's name, city, and signup date. No other columns.

```
SELECT name, city, signup_date
FROM customers
ORDER BY signup_date;
```

Notice that the newest customers appear last. Change ORDER BY to ORDER BY signup_date DESC and see what happens.

Query 3 — Filter by status

Find all orders that have been delivered.

```
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE status = 'delivered'
ORDER BY order_date DESC;
```

How many delivered orders are there? Now change 'delivered' to 'pending'. How many pending orders are there? Try 'cancelled'. Understanding the distribution of values in a status column is standard early-stage data exploration.

Query 4 — Filter by price range

Find all products priced between 1000 and 5000 rupees. Write it two ways and confirm both give identical results.

```
-- Method A: BETWEEN
SELECT product_name, category, price
FROM products
WHERE price BETWEEN 1000 AND 5000
ORDER BY price;

-- Method B: comparison operators
SELECT product_name, category, price
FROM products
WHERE price >= 1000 AND price <= 5000
ORDER BY price;
```

They should return the same rows. BETWEEN is inclusive — a product priced at exactly 1000 or exactly 5000 is included in both.

Query 5 — Top 10 highest-value orders

List the 10 highest-value orders with their order ID, customer ID, total amount, and status.

```
SELECT order_id, customer_id, total_amount, status
FROM orders
ORDER BY total_amount DESC
LIMIT 10;
```

Notice that high-value orders appear across different statuses — some delivered, some pending. This is the kind of thing a fraud detection team would look at. You will run EXPLAIN ANALYZE on this query in Part 4.

Query 6 — Multi-condition filter

Find all delivered orders placed in 2025 with a total above 10,000 rupees. Sort by total, highest first.

```
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE status = 'delivered'
  AND order_date >= '2025-01-01'
  AND total_amount > 10000
ORDER BY total_amount DESC;
```

Each condition is on its own line, aligned on AND. This is not required but makes complex filters much easier to read and debug. Get into this habit now.

Query 7 — Pattern matching on email

Find all customers with a Gmail address.

```
SELECT name, email, city
FROM customers
WHERE email LIKE '%@gmail.com'
ORDER BY name;
```

Now try replacing LIKE with ILIKE (case-insensitive version). Do you get different results? Why or why not? Try `LIKE '%yahoo%'` to find Yahoo users. Then try `LIKE 'a%'` — what does that return?

Query 8 — NULL handling (unshipped orders)

Find all orders that have not yet shipped — meaning the shipped_date column has no value.

```
SELECT order_id, customer_id, order_date, status, total_amount
FROM orders
WHERE shipped_date IS NULL
ORDER BY order_date;
```

Now deliberately try the wrong approach to see why it fails:

```
-- This returns zero rows — not an error, just silently wrong
SELECT * FROM orders WHERE shipped_date = NULL;
```

NULL is not a value — it is the absence of a value. Nothing equals NULL, not even NULL itself. IS NULL is the only correct check.

Query 9 — Computed column

Display each product's name, category, original price, and a discounted price column showing a 20% discount. Also add a column showing the discount amount in rupees. Round both to 2 decimal places.

```
SELECT product_name,
       category,
       price                              AS original_price,
       ROUND(price * 0.80, 2)             AS discounted_price,
       ROUND(price * 0.20, 2)             AS you_save
FROM products
ORDER BY discounted_price DESC;
```

ROUND takes two arguments: the value and the number of decimal places to keep. You can do any arithmetic directly in SELECT on numeric columns — addition, subtraction, multiplication, division.

Query 10 — Bring everything together

Find the top 5 highest-value unshipped orders from 2025. Add a column called priority that shows 'URGENT' for orders above 10,000 and 'NORMAL' for the rest.

```
SELECT order_id,
       customer_id,
       total_amount,
       order_date,
       status,
       CASE
           WHEN total_amount > 10000 THEN 'URGENT'
           ELSE 'NORMAL'
       END AS priority
FROM orders
WHERE shipped_date IS NULL
  AND order_date >= '2025-01-01'
ORDER BY total_amount DESC
LIMIT 5;
```

CASE WHEN is SQL's conditional logic — like an if/else statement. Read it as: "For each row, check the condition. If it is true, use that value. If nothing matches, use ELSE." You will use this constantly in real data work.

Now extend it yourself: add a third tier. Orders above 20,000 are 'CRITICAL', orders between 5,000 and 20,000 are 'URGENT', and everything else is 'NORMAL'. Hint: CASE WHEN supports multiple WHEN clauses.

Save Your Work

After writing all 10 queries, save them to your file and commit:

```
# In Ubuntu terminal
cd ~/database-labs
git add lab2/
git commit -m "Lab 2: Completed 10 SQL exercises on e-commerce dataset"
git push origin main
```

Part 4 — Performance Awareness

You will optimise queries properly in Week 6. But you should start building instincts now. The tool is EXPLAIN ANALYZE.

Prefix any query with EXPLAIN ANALYZE and PostgreSQL will run it, then tell you how it executed and how long it took.

```
EXPLAIN ANALYZE
SELECT order_id, customer_id, total_amount, status
FROM orders
ORDER BY total_amount DESC
LIMIT 10;
```

Run this in DBeaver's SQL editor. Look at the output and find two things:

Seq Scan vs Index Scan. A Seq Scan means the database read every row in the table — like reading a book cover to cover to find one sentence. An Index Scan means it jumped directly to relevant rows using an index. On small tables like yours, both are fast. On a table with 10 million rows, the difference is the gap between 2 milliseconds and 4 minutes.

actual time. The output shows time in milliseconds. Your queries on this small dataset will be very fast. Note the number anyway — in your report, you will be asked to reason about what happens to that number as data grows.

For your deliverable, paste the full EXPLAIN ANALYZE output for Query 5 and answer: is it a Seq Scan or an Index Scan, and how many milliseconds did it take?

Habits to build now that protect you on large datasets:

Never run SELECT * on a table you do not know — use LIMIT 10 first. Always filter with WHERE when you only need a subset of rows. Avoid LIKE patterns that start with % (like `LIKE '%gmail%'`) on large tables — the database cannot use an index for these and must scan every row. Always use LIMIT when you only need top-N results.

Part 5 — AI Prompting for SQL

AI tools are genuinely useful for learning SQL, but only if you use them in a way that produces actual understanding rather than code you cannot explain. The rule from Week 1 still applies: if someone asked you to explain this query, could you?

Three prompting patterns that work:

Pattern 1 — Explain a query you do not fully understand

Use this when AI generates something, or when you find a query online, and parts of it are unclear.

```
I have this SQL query and I am not sure how part of it works:

SELECT order_id,
       customer_id,
       total_amount,
       CASE
           WHEN total_amount > 10000 THEN 'URGENT'
           ELSE 'NORMAL'
       END AS priority
FROM orders
WHERE shipped_date IS NULL
ORDER BY total_amount DESC
LIMIT 5;

My table is called orders with columns: order_id, customer_id,
order_date, status, total_amount, shipped_date.

Please:
1. Explain the CASE WHEN block line by line in plain terms
2. What would happen if I removed the ELSE clause entirely?
3. How would I add a third tier — orders above 20000 as 'CRITICAL'?
4. Suggest one small change I can make to experiment and learn
```

Pattern 2 — Debug an error

Paste the exact error and the exact query. Vague descriptions produce vague help.

```
I am running this query in PostgreSQL and getting an error:

SELECT * FROM orders WHERE order_date > 2025-01-01;

Error: ERROR: operator does not exist: date = integer

Please:
1. Explain exactly why this error occurs
2. Show me the corrected query
3. Explain the rule so I do not make this mistake again
4. What other data type mistakes are common in SQL with dates?
```

Pattern 3 — Extend a query you already wrote

Use this after you have a working query that you understand. Ask for the next level.

```
I wrote this query myself and I understand how it works:

SELECT product_name, category, price
FROM products
WHERE price > 2000
ORDER BY price DESC
LIMIT 10;

My products table has columns: product_id, product_name, category,
price, stock_qty.

Help me learn by:
1. Explaining how to add a column showing stock value (price × stock_qty)
   — explain the concept first, then show me the code
2. Showing me how to filter to only show products where stock is below 20
3. Giving me a challenge variation I can attempt on my own first
```

Good vs bad prompts:

Bad	Good
"Write SQL to get top orders"	"Write SQL to find the top 10 orders by total_amount where status is delivered. Explain each clause."
"Fix my query"	"I get this error: [exact error]. Here is my query: [exact query]. Explain why and fix it."
"What is CASE WHEN"	"Explain CASE WHEN with a concrete example using an orders table with a status column"

The pattern in every good prompt is the same: give context (your exact table structure), provide your exact code or exact error, and always ask for an explanation alongside the answer.

Part 6 — Deliverables

Submit a single PDF named `Lab02_FirstNameLastName_RollNumber.pdf` to Google Classroom by the announced deadline. Late penalty: **Zero Marks!**

Section 1 — 10 Queries (40 points)

For each query: the formatted SQL code, a screenshot of the results from DBeaver or psql, and one or two sentences in plain English explaining what the output shows. Do not just describe what the query does — describe what the results tell you about the data.

Section 2 — Performance Report (20 points)

Run EXPLAIN ANALYZE on Query 5 and paste the full output. Write a short paragraph answering: is it a Seq Scan or an Index Scan? How many milliseconds did it take? If this table had 5 million rows instead of 30, what do you think would change?

Section 3 — AI Learning Log (30 points)

Minimum 3 entries using the template below. Quality over quantity. An entry that shows you modified something, tested it, understood why it worked, and learned something is worth far more than three entries that just paste AI responses and call it done.

```
AI INTERACTION #[number]
═══════════════════════════════════════════

Date:
AI Tool:  Claude / ChatGPT

TASK
What were you trying to accomplish?

PROMPT USED
Paste the exact prompt you sent.

RESPONSE QUALITY
Rating: ⭐⭐⭐⭐☆   Why helpful or not?

KEY LEARNINGS
In your own words — not the AI's words.
What did you actually understand from this?

HOW I VERIFIED
Did you run the query? Did results match what AI predicted?

WHAT I MODIFIED
Show your version. What did you change and why?
```

Section 4 — Reflection (10 points)

150 words minimum. What surprised you about how SQL works? What clicked? What is still confusing? What do you want to understand better before Week 3?

Quick Reference

```
-- ── Query skeleton ──────────────────────────────────────
SELECT  column1,
        column2         AS alias,
        price * 0.85    AS discounted
FROM    table_name
WHERE   condition1
  AND   condition2
ORDER BY column1 DESC
LIMIT   10;

-- ── Conditional column ──────────────────────────────────
CASE
    WHEN total_amount > 20000 THEN 'CRITICAL'
    WHEN total_amount > 10000 THEN 'URGENT'
    ELSE 'NORMAL'
END AS priority

-- ── Common functions ────────────────────────────────────
ROUND(price, 2)                   -- round to 2 decimal places
UPPER(name)                       -- convert to uppercase
LOWER(email)                      -- convert to lowercase
LENGTH(product_name)              -- count characters
COALESCE(shipped_date, 'N/A')     -- replace NULL with a default
TO_CHAR(order_date, 'YYYY-MM')    -- format date as text
EXTRACT(YEAR FROM order_date)     -- pull year from a date

-- ── psql quick commands ─────────────────────────────────
\dt                 -- list all tables
\d table_name       -- describe a table's structure
\timing             -- toggle query timing on/off
\i path/to/file     -- run a SQL file
\q                  -- quit psql
```

Submission Checklist

Before submitting, verify:

- [ ] ecommerce_setup.sql and queries.sql are committed and pushed to your GitHub repo
- [ ] PDF includes all 10 queries with screenshots and plain-English explanations
- [ ] EXPLAIN ANALYZE output is included with a written interpretation
- [ ] AI Learning Log has minimum 3 genuine entries showing learning, not copying
- [ ] Reflection is at least 150 words and is actually reflective
- [ ] File is named correctly: Lab02_FirstNameLastName_RollNumber.pdf
- [ ] GitHub repo URL is included somewhere in the document

Next lab — Week 3: GROUP BY, aggregate functions, window functions, and CTEs. You will use the same e-commerce dataset, so make sure you understand every query from this week before arriving.